1.This Java program demonstrates **single inheritance**.

The program has three classes: Add, Subtract, and Single.

Add **Class**

The Add class is the **parent class**.

It has a variable a with the value 10. It also contains a method called demo().

Inside the demo() method, two local variables, x and y, are created with values 20 and 30. These values are added together with a . The result is then displayed.

Subtract **Class**

The Subtract class is the **child class** because it extends the Add class.

By using inheritance, Subtract can access the variable a and the method demo() from the Add class.

The Subtract class also has its own variable b, whose value is 5.

It contains a method called sample(). This method subtracts b from the inherited variable a and the result is displayed.

Single **Class**

The Single class contains the main() method, which is the starting point of the program.

An object of the Subtract class is created. Since Subtract inherits from Add, this object can access both its own methods and the inherited methods.

First, the sample() method is called, which performs subtraction.

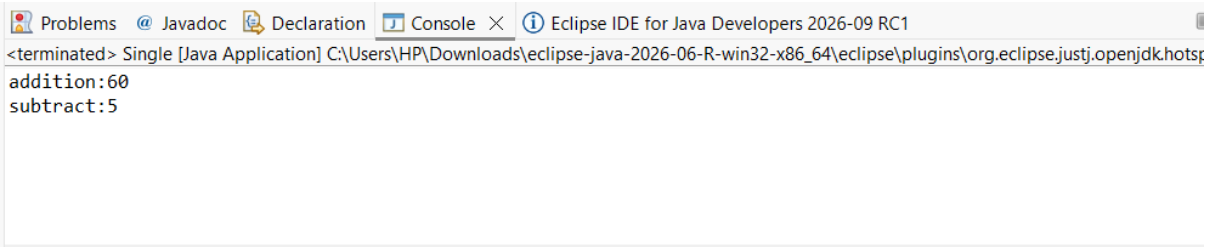
Then, the demo() method is called, which is inherited from the Add class and performs addition.

Code:

```
package basics;

class Add {

  int a = 10;


  void demo() {

    int x = 20, y = 30;

    System.out.println("addition:" + (a + x + y));
```

```
    }
}

class Subtract extends Add {
    int b = 5;

    void subtract() {
        System.out.println("subtract:" + (a - b));
    }
}

public class Single {
    public static void main(String args[]) {
        Subtract s = new Subtract();
        s.demo();
        s.subtract();

    }

}
```

Output:



2.

In this code, C2 and C3 are two separate classes, and C1 is trying to inherit properties and methods from both classes using extends. However, Java does not support multiple inheritance using classes, so a class can extend only one parent class. Therefore, class C1 extends C2, C3 is invalid and produces a compile-time error. This restriction avoids ambiguity when both parent classes contain methods with the same name.

Correct alternative using interface:

```
interface C2 {

}


interface C3 {

}


class C1 implements C2, C3 {

}
```

3.This program demonstrates the concept of **multiple inheritance using interfaces** in Java. Two interfaces are created, each declaring one or more abstract methods. The first interface also contains constant variables. A class then implements both interfaces, meaning it must provide implementations for all the methods declared in the interfaces.

The implemented methods perform simple arithmetic operations using the interface constants. One method multiplies a constant value and displays the result, while the other subtracts one constant from another and prints the output.

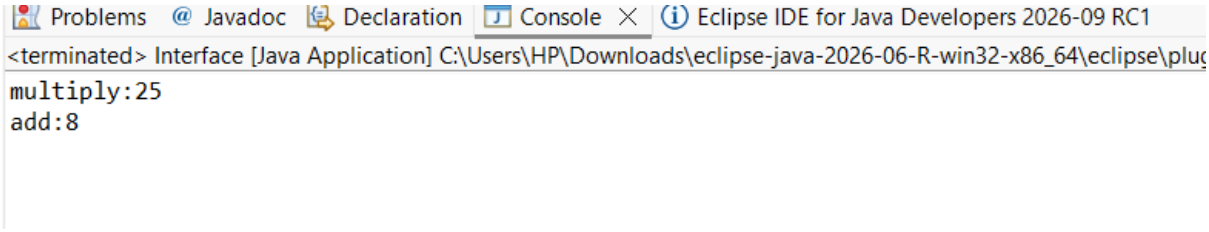
In the main method, an object of the implementing class is created, and the implemented methods are called. When the program runs, it executes these methods and displays the results of the multiplication and subtraction operations.

Overall, this program shows how a single class can implement multiple interfaces, access interface constants, override abstract methods, and use an object to invoke those methods. It is a simple example of achieving multiple inheritance in Java through interfaces.

Code:

```
interface I1{
   int x=5;
   int y=3;
   void sample();

   }
class Project implements I1{
   @Override
   public void sample() {
 System.out.println("multiply:" +(x*5));

   }



}
public class Interface {
   public static void main(String[] args) {
      Project p =new Project();
      p.sample();
}
}
```

Output:



4.

This Java program demonstrates **multiple inheritance using interfaces**. In Java, a class cannot extend multiple classes, but it can implement multiple interfaces.

I1 **Interface**

The first interface, I1, contains two variables, x and y, with values 5 and 3.

It also declares a method called sample().

Since these variables are declared inside an interface, they are automatically **public, static, and final**. Therefore, their values cannot be changed.

I2 **Interface**

The second interface, I2, contains a method called subtract().

This method is abstract, so it does not have a body in the interface. The class that implements I2 must provide the method's implementation.

Project **Class**

The Project class implements both I1 and I2.

Therefore, it must provide implementations for both sample() and subtract() methods.

The sample() method performs multiplication. It uses the value of x, which is 5, and multiplies it by 5.

Code: interface I1{

```
    int x=5;

    int y=3;

    void sample();
```

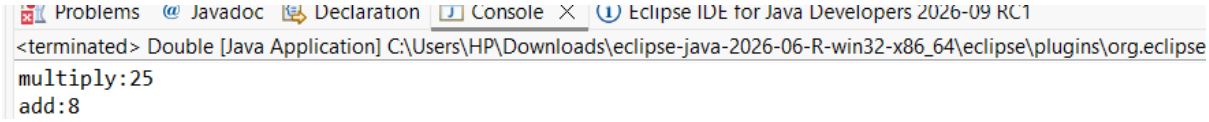
}

interface I2{

```
        void subtract();
```

```
	}

class Project implements I1,I2{
	@Override
	public void sample() {
 System.out.println("multiply:" +(x*5));

	}

	@Override
	public void subtract() {
		System.out.println("subtract :" +(x-y));
	}

}
public class Double {
	public static void main(String[] args) {
		Project p =new Project();
		p.sample();
		p.subtract();
	}

}
```

Output:



5.

In this code, I1 is an interface and C2 is a class. The statement class C1 implements C2 extends I1 is invalid because the keywords are used incorrectly. A class can extend only another class, so C1 should use extends C2. A class can implement an interface, so C1 should use implements I1. Therefore, the correct statement is class C1 extends C2 implements I1. This means C1 inherits from class C2 and implements the interface I1.

C1 implements C2 extends I1 is invalid

6.

This Java program demonstrates **interface inheritance** and **interface implementation**.

Animal **Interface**

The Animal interface contains a method called sound().

The method is declared but does not have a body because it is an abstract method.

So, any class that implements Animal must provide an implementation for sound().

Dog **Interface**

The Dog interface **extends** the Animal interface.

This means Dog inherits the sound() method from Animal.

The Dog interface declares sound() again. Since Dog extends Animal, it can provide its own declaration of the same method.

The relationship is:

Animal → Dog

Here, Animal is the parent interface and Dog is the child interface.

Interface1 **Class**

The Interface1 class implements the Dog interface.

Because Dog inherits from Animal, the Interface1 class must provide an implementation for the sound() method.

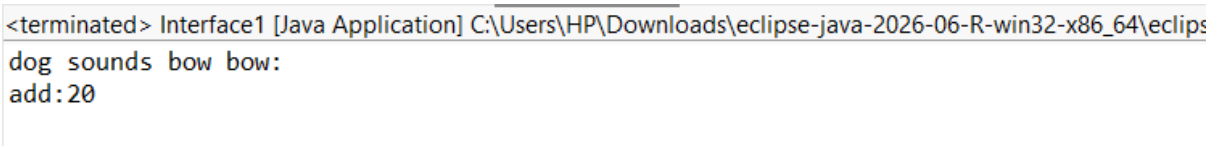
The sound() method prints

Code:

```
interface Animal{
   void sound();

}
interface Dog extends Animal{
   public void sound();



   }



public class Interface1 implements Dog {
void sample(){
   int a=10;
   System.out.println("add:"+(a+10));
}

   @Override
   public void sound() {
      System.out.println("dog sounds bow bow: ");

   }

   public static void main(String[] args) {

   Interface1 i=new Interface1();
   i.sound();
```

```
    i.sample();
}
}
```

Output:



7.

In this code, C1 is a class and I1 is an interface. The statement interface I1 implements C1 is invalid because an interface cannot implement a class. The keyword implements is used by a class to implement an interface. An interface can use the keyword extends to inherit from another interface.

I1 implements C1 is invalid

8.

This Java program demonstrates **multiple inheritance using interfaces**. It shows how one interface can extend more than one interface and how a class can implement the combined interface.

Cat **Interface**

The Cat interface contains one abstract method called eat().

This method does not have a body. Any class that implements Cat must provide the implementation of eat().

Dog **Interface**

The Dog interface contains one abstract method called sound().

Any class implementing Dog must provide the implementation of sound().

Animals **Interface**

The Animals interface extends **both** Cat **and** Dog.

This means Animals inherits the methods from both interfaces:

- eat() from Cat

- sound() from Dog

It declares both methods again.

The relationship is:

Cat + Dog → Animals

This is an example of **multiple inheritance through interfaces**.

Interface2 **Class**

The Interface2 class implements the Animals interface.

Because Animals extends both Cat and Dog, the Interface2 class must implement both eat() and sound().

eat() **Method**

The eat() method prints two statements:

- It says that the dog eats bones.
- It says that the cat drinks milk

Code:

```
interface Cat{
   void eat();
}
interface Dog {
   void sound();

}
interface Animals extends Cat,Dog{
void eat();

   void sound();

}
```

```
public class Interface2 implements Animals{

   @Override
   public void eat() {
      System.out.println("dog eats bones:");
      System.out.println("cat drinks milk:");
   }

   @Override
   public void sound() {
      System.out.println("dog sounds bow bow");
      System.out.println("cat sounds meow meow:")
}
   public static void main(String[] args) {
      Interface2 i=new Interface2();
      i.sound();
      i.eat();
   }
 }
```

Output: